

Natural Language Processing with Recurrent Neural Networks (RNNs)

A comprehensive educational repository covering the fundamentals of **Natural Language Processing (NLP)** using **Recurrent Neural Networks (RNNs)**, including sequence modeling, LSTMs, GRUs, practical concepts, mathematical foundations, and learning resources.

Table of Contents

- Introduction
 - What is Natural Language Processing?
 - Why RNNs?
 - Sequential Data
 - Recurrent Neural Networks
 - Hidden States
 - Mathematical Foundations
 - Unrolling an RNN
 - Word Embeddings
 - Tokenization
 - Vocabulary
 - Padding & Sequence Length
 - Training an RNN
 - Backpropagation Through Time (BPTT)
 - Vanishing & Exploding Gradients
 - Long Short-Term Memory (LSTM)
 - Gated Recurrent Unit (GRU)
 - RNN Architectures
 - NLP Applications
 - Loss Functions
 - Optimizers
 - Evaluation Metrics
 - Python Libraries
 - NLP in Healthcare & Bioinformatics
 - RNNs vs Transformers
 - Learning Roadmap
 - Resources
-

Introduction

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to understand, interpret, analyze, and generate human language.

Traditional NLP systems relied on handcrafted rules and statistical methods, while modern NLP combines deep learning architectures such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM), Gated Recurrent Units (GRU), and more recently Transformers.

This repository focuses on understanding RNN-based NLP, providing the theoretical foundation required before studying Transformer models and Large Language Models (LLMs).



What is NLP?

Natural Language Processing combines:

- Artificial Intelligence
- Machine Learning
- Deep Learning
- Computational Linguistics

Its goal is to allow machines to process human language similarly to humans.

Common Applications

- Sentiment Analysis
 - Chatbots
 - Machine Translation
 - Question Answering
 - Text Summarization
 - Spam Detection
 - Search Engines
 - Speech Recognition
 - Medical Report Analysis
 - Clinical Decision Support
-



Why Recurrent Neural Networks?

Unlike traditional neural networks, RNNs are designed to process sequential information.

Example:

The → patient → has → diabetes

Each word depends on previous words.

An RNN maintains a memory of previous inputs using a **Hidden State**, allowing it to capture contextual information.



Sequential Data

A sequence is an ordered collection of elements.

Examples include:

- Natural language sentences
- DNA sequences
- RNA sequences
- Protein sequences
- ECG signals
- Time-series data
- Speech
- Music
- Video frames



Recurrent Neural Networks (RNNs)

An RNN processes one element of a sequence at a time while carrying forward information from previous steps.

Input → Hidden State → Output
 ↑
Previous Memory

Unlike feedforward neural networks, the hidden state acts as the network's memory.



Mathematical Foundations

Hidden state computation:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

Output:

$$y_t = W_y h_t$$

Where:

- x_t : Current input
- h_{t-1} : Previous hidden state
- W_h : Hidden weights
- W_x : Input weights
- W_y : Output weights

- b : Bias term
-

Unrolling an RNN

```
x1 → h1 → y1
      ↓
x2 → h2 → y2
      ↓
x3 → h3 → y3
      ↓
x4 → h4 → y4
```

The same parameters are reused at every timestep.

Text Preprocessing

Tokenization

Converts text into tokens.

```
"I love AI"
      ↓
["I", "love", "AI"]
```

Vocabulary

A vocabulary is the collection of all known words.

```
patient
doctor
hospital
medicine
AI
```

Each token receives a numerical index.

Word Embeddings

Instead of One-Hot Encoding, words are represented using dense vectors.

Popular embedding techniques:

- Word2Vec
- GloVe
- FastText

These embeddings capture semantic similarity between words.

Padding

Neural networks require fixed-length sequences.

```
I love AI
↓
I
love
AI
<PAD>
<PAD>
```

Training an RNN

Pipeline:

```
Text
↓
Tokenizer
↓
Embedding Layer
↓
RNN
↓
Dense Layer
↓
Prediction
↓
Loss Function
```

↓
Backpropagation Through Time

Training repeats across multiple epochs until convergence.



Backpropagation Through Time (BPTT)

RNNs use a specialized version of backpropagation that unfolds the network across time.

Errors propagate backward through every timestep, allowing the model to update shared weights.



Challenges of Vanilla RNNs

Vanishing Gradient

The model gradually forgets information from earlier timesteps.

Example:

John moved to Paris.

...

He enjoys the city.

The network may lose the connection between "He" and "John".

Exploding Gradient

Gradients become excessively large during training, causing unstable optimization.



Long Short-Term Memory (LSTM)

LSTM networks address the limitations of standard RNNs by introducing:

- Cell State
- Hidden State

Three gates:

- Forget Gate

- Input Gate
- Output Gate

These gates regulate what information is stored, updated, and discarded.

Gated Recurrent Unit (GRU)

GRUs simplify LSTMs while maintaining strong performance.

Components:

- Update Gate
- Reset Gate

Advantages:

- Faster training
 - Fewer parameters
 - Competitive accuracy
-

RNN Architectures

One-to-One

Classification

Image → Label

One-to-Many

Image Captioning

Image → Sentence

Many-to-One

Sentiment Analysis

Sentence → Positive

Many-to-Many

Machine Translation

NLP Applications

- Sentiment Analysis
 - Spam Detection
 - Machine Translation
 - Named Entity Recognition
 - Part-of-Speech Tagging
 - Text Classification
 - Speech Recognition
 - Text Generation
 - Chatbots
 - Clinical NLP
 - Medical Report Classification
 - DNA Sequence Analysis
-

Loss Functions

Classification

- Cross-Entropy Loss

Regression

- Mean Squared Error (MSE)
-

Optimizers

- SGD
 - Adam
 - AdamW
 - RMSProp
-

Evaluation Metrics

Classification:

- Accuracy
- Precision
- Recall

- F1-Score
- ROC-AUC

Language Generation:

- BLEU
- ROUGE
- METEOR
- Perplexity

Popular Libraries

- Python
- NumPy
- Pandas
- PyTorch
- TensorFlow
- Keras
- Hugging Face Transformers
- spaCy
- NLTK
- Gensim

NLP in Healthcare & Bioinformatics

NLP plays an increasingly important role in biomedical research.

Applications include:

- Electronic Health Record analysis
- Clinical note classification
- Disease prediction
- Medical report summarization
- Biomedical literature mining
- Drug discovery
- DNA/RNA sequence modeling
- Protein sequence prediction
- Medical conversational agents

RNN vs Transformer

RNN	Transformer
Sequential computation	Parallel computation

RNN	Transformer
Limited memory	Global attention
Slower training	Faster training
Vanishing gradient	Better long-range modeling
Foundation of sequence learning	Current state-of-the-art



Learning Roadmap

1. Python Programming
2. Mathematics for AI
3. Machine Learning
4. Deep Learning
5. Text Preprocessing
6. RNN
7. LSTM
8. GRU
9. Sequence-to-Sequence Models
10. Attention Mechanisms
11. Transformers
12. Large Language Models (LLMs)



Recommended Resources

Books

- *Speech and Language Processing* — Jurafsky & Martin
- *Dive into Deep Learning*
- *Natural Language Processing with PyTorch*

Courses

- DeepLearning.AI — NLP Specialization
- Stanford CS224N
- fast.ai
- Carnegie Mellon NLP

YouTube

- Andrew Ng
- Aladdin Persson
- Krish Naik
- Yannic Kilcher



Suggested Projects

Beginner

- Spam Email Classifier
- Movie Review Sentiment Analysis
- News Classification

Intermediate

- Character-Level Text Generator
- Next Word Prediction
- Language Detection

Advanced

- Machine Translation
 - Medical Report Classifier
 - DNA Sequence Classifier
 - Biomedical Question Answering
-



Learning Outcomes

After completing this repository, you will be able to:

- Understand the fundamentals of Natural Language Processing.
 - Explain how Recurrent Neural Networks process sequential data.
 - Implement RNNs, LSTMs, and GRUs using PyTorch or TensorFlow.
 - Build NLP models for classification and sequence generation.
 - Understand the limitations of RNNs and why Transformers became the dominant architecture.
 - Apply sequence modeling techniques to healthcare and bioinformatics applications.
-



Support

If you find this repository useful, consider giving it a ★ to support the project and help others discover it.